
Project Specification: AI-Powered Cyber Incident Monitoring Tool

Daniel Vetrila (C00271021)

27 November 2024

Contents

1. Introduction	2
1.1. Project Overview	2
1.2. Goals and Objectives	2
1.3. Scope	2
2. Requirements	4
2.1. Functional Requirements	4
2.2. Non-Functional Requirements	5
2.3. System Requirements	5
2.4. User Roles	6
3. System Architecture and Design	7
3.1. High-Level Architecture Diagram	7
3.2. Key Components	7
3.3. Technology Stack	8
3.4. Data Flow Diagram	9
4. Implementation Strategy	11
4.1. Development Methodology	11
4.2. Modules and Phases	11
5. Project Plan	13
5.1. Milestones and Deliverables	13
5.2. Timeline	13
5.3. Resource Allocation	14
6. Metrics for Success (FURPS+)	15
6.1. Functionality	15
6.2. Usability	15
6.3. Reliability	15
6.4. Performance	16
6.5. Supportability (Maintainability)	16
6.6. Security (Added Metric)	17
7. Conclusion	18

1. INTRODUCTION

1.1. Project Overview

This document outlines the specification for the **AI-Powered Cyber Incident Monitoring Tool**. The project aims to develop a comprehensive system for automated collection, analysis, enrichment, and reporting of cybersecurity threats. It leverages cutting-edge technologies, including Artificial Intelligence (via Google's Gemini API), threat intelligence feeds (ThreatFox), and established cybersecurity frameworks (MITRE ATT&CK) to provide timely, actionable insights to security professionals. The tool will feature both a web-based graphical user interface (GUI) for interactive analysis and a command-line interface (CLI) for automation and scripting.

1.2. Goals and Objectives

The primary goal of this project is to enhance an organization's capability to detect, understand, and respond to cyber threats efficiently and effectively.

Key Objectives:

- **Automate Threat Data Collection:** Continuously gather threat-related information from diverse online sources (news articles, RSS feeds).
- **Leverage Gemini AI for IOC Identification & Analysis:** Utilize AI to intelligently extract Indicators of Compromise (IOCs) from collected data and provide insightful analysis.
- **Enrich IOC Data:** Integrate with the ThreatFox API to enrich identified IOCs with known threat intelligence.
- **Contextualize Threats with MITRE ATT&CK:** Map identified threats and IOCs to the MITRE ATT&CK framework to understand adversarial tactics, techniques, and procedures (TTPs).
- **Implement Threat Prioritization:** (Future Scope/Refinement) Develop a mechanism to rank threats based on a configurable organizational risk profile.
- **Generate Automated, Actionable Reports:** Produce clear, concise, and comprehensive reports on analyzed threats, including visualizations and recommended actions.
- **Provide Multi-Platform Access:** Offer both a user-friendly web interface for visual data exploration and a robust CLI for advanced users and automation.
- **Enhance Incident Response Efficiency:** Reduce the manual effort and time required for threat analysis and reporting.
- **Ensure Scalability and Maintainability:** Design the system with modularity and best practices to allow for future enhancements and easy maintenance.

1.3. Scope

In Scope:

- Development of data collection modules for specified web sources (e.g., NewsNow, ThreatPost RSS).
- Integration with Google Gemini AI for IOC extraction from text and AI-driven analysis.
- Integration with ThreatFox API for IOC enrichment.
- Mapping of threat data to the MITRE ATT&CK framework (utilizing a local cache of MITRE data).
- Development of a web interface (using FastAPI, Jinja2) to display:

- Monitored articles and extracted IOCs.
- Detailed analysis reports for IOCs and articles.
- Real-time updates via WebSockets.
- Manual IOC submission and analysis.
- PCAP file analysis for IOC presence.
- Development of a basic CLI for:
 - IOC analysis and report generation.
 - (Potentially) Triggering article monitoring tasks.
- Automated HTML report generation for IOCs.
- Local caching mechanism for MITRE ATT&CK data.
- Basic logging and error handling.

Out of Scope (for initial version):

- Advanced user authentication and role-based access control (RBAC) for the web interface.
 - Sophisticated, customizable threat ranking based on detailed organizational profiles.
 - Active blocking or remediation capabilities.
 - Distributed deployment and high-availability clustering.
 - Machine learning model training for custom IOC detection (relies on Gemini for this).
 - Extensive support for a wide array of data sources beyond the initially specified ones.
-

2. REQUIREMENTS

2.1. Functional Requirements

Table 1: Functional Requirements

ID	Requirement Description	Priority	Justification
FR01	The system shall automatically collect articles from pre-defined web sources (News-Now, ThreatPost RSS).	High	Core function for obtaining raw threat intelligence.
FR02	The system shall extract potential IOCs (IPs, URLs, domains, file hashes) from collected articles using regex and AI.	High	Essential for identifying actionable threat indicators.
FR03	The system shall query the ThreatFox API to enrich identified IOCs with available threat data.	High	Provides crucial context and details about known threats.
FR04	The system shall utilize the Gemini AI API to perform detailed analysis of IOCs and generate structured summaries.	High	Provides advanced analytical capabilities and human-readable insights.
FR05	The system shall utilize the Gemini AI API to analyze full article content and provide structured summaries.	Medium	Enhances understanding of broader threat contexts from articles.
FR06	The system shall map enriched IOC data to relevant MITRE ATT&CK techniques and tactics.	High	Standardizes threat description and aids in understanding attacker methodologies.
FR07	The system shall generate HTML reports for analyzed IOCs, including AI summary, ThreatFox data, and MITRE mapping.	High	Core deliverable for presenting findings to users.
FR08	The web interface shall display a dashboard of recently monitored articles and their status.	High	Provides an overview of current threat landscape being monitored.
FR09	The web interface shall allow users to click on an article to view its content and any AI-generated analysis.	High	Enables users to drill down into specific threat information.
FR10	The web interface shall allow users to manually submit an IOC for analysis.	High	Provides flexibility for analyzing specific IOCs not automatically picked up.
FR11	The web interface shall display the detailed HTML report for any analyzed IOC.	High	User access to analysis results.
FR12	The web interface shall use WebSockets to provide real-time updates for new articles and analyses.	Medium	Enhances user experience by providing timely information without manual refreshes.
FR13	The system shall allow checking for the presence of a given IOC within a provided PCAP/PCAPNG file.	Medium	Adds capability to correlate external threats with local network activity.
FR14	The system shall maintain a local cache of the MITRE ATT&CK enterprise framework data, refreshable periodically.	Medium	Improves performance and reduces external dependency for MITRE mapping.
FR15	The CLI shall allow users to submit an IOC for analysis and receive a report (text or path to HTML).	Medium	Caters to users preferring command-line operations or scripting.

2.2. Non-Functional Requirements

Table 2: Non-Functional Requirements

ID	Requirement Description	Type	Priority	Justification
NFR01	The web interface shall be responsive and accessible on modern web browsers (Chrome, Firefox, Edge).	Usability	High	Ensures a good user experience across common platforms.
NFR02	The system should process and analyze a typical IOC within 60 seconds (excluding external API latency).	Performance	Medium	Provides timely results for user queries.
NFR03	The article monitoring component should check for new articles at a configurable interval (default: 10 minutes).	Performance	Medium	Balances timeliness with resource usage and API rate limits.
NFR04	The system shall be designed modularly to facilitate future enhancements and maintenance.	Maintainability	High	Ensures long-term viability and ease of development.
NFR05	API keys and sensitive configurations shall be managed securely (e.g., via environment variables or a .env file).	Security	High	Protects sensitive credentials.
NFR06	The system shall handle external API failures gracefully (e.g., retries, error messages) without crashing.	Reliability	High	Ensures system stability when external dependencies are unavailable.
NFR07	Code shall be well-documented with comments explaining complex logic.	Maintainability	Medium	Aids understanding and future development.
NFR08	The system should be deployable on a standard Linux environment with Python 3.8+ installed.	Deployability	High	Ensures compatibility with common deployment environments.

2.3. System Requirements

- **Operating System:** Linux (recommended), macOS, Windows (with Python environment).
- **Python Version:** 3.8 or higher.
- **Dependencies:** As listed in `requirements.txt` (Requests, FastAPI, Uvicorn, Jinja2, Google Generative AI SDK, python-dotenv, stix2, BeautifulSoup, etc.).
- **External Tools:** `tshark` for PCAP analysis (must be in system PATH).
- **Internet Access:** Required for accessing news sources, ThreatFox API, and Gemini AI API.
- **API Keys:**
 - Google Gemini AI API Key.
 - (No API key explicitly required for ThreatFox public API, but subject to their terms of use).
- **Hardware (Development/Testing):**

- Minimum 2 CPU cores.
- Minimum 4GB RAM (8GB recommended for smoother AI interactions and multiple processes).
- Minimum 10GB free disk space (for OS, project files, dependencies, and MITRE cache).

2.4. User Roles

1. Cybersecurity Analyst (Primary User):

- Uses the web interface to monitor threats, view analyses, and investigate IOCs.
- May use the CLI for quick lookups or specific automation tasks.
- Benefits from clear reports and MITRE ATT&CK mapping for understanding threats.

2. Incident Responder:

- Uses the tool to quickly gather intelligence on IOCs related to an active incident.
- Leverages PCAP analysis feature to check for local presence of threats.

3. Threat Intelligence Analyst:

- Uses the tool to gather and contextualize emerging threats from public sources.
 - May use CLI for scripting bulk IOC lookups or integrating with other tools.
-

3. SYSTEM ARCHITECTURE AND DESIGN

3.1. High-Level Architecture Diagram

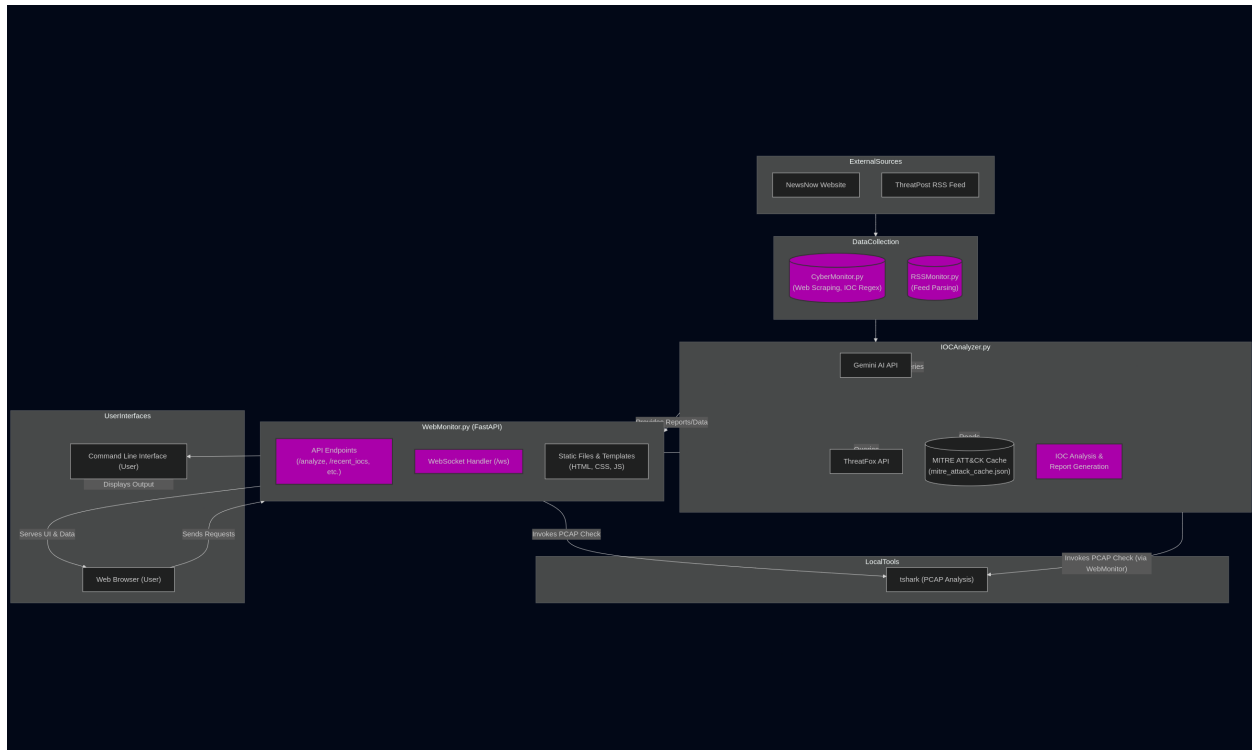


Diagram Note

Zoom in to see the details.

3.2. Key Components

1. ``ioc\_analyzer.py`` (Core Logic):

- `IOCAalyzer` class: Manages IOC analysis workflow.
- `_setup_gemini()`: Configures Gemini AI client.
- `_setup_mitre()`: Loads/caches MITRE ATT&CK data.
- `search_threatfox(ioc)`: Fetches IOC data from ThreatFox.
- `map_to_mitre(ioc_data)`: Maps threat data to MITRE techniques.
- `analyze_with_gemini(ioc_data)`: Sends data to Gemini for analysis and structuring.
- `generate_report(ioc)`: Orchestrates the full analysis for an IOC.
- `display_report(report)`: Formats the analysis into an HTML report.
- `analyze_article_content(content)`: Uses Gemini to analyze full article text.

2. ``cyber\_monitor.py`` (Article Monitoring):

- **CyberMonitor class:** Handles fetching and processing articles from news sources.
- **get_articles():** Scrapes articles from configured URLs (e.g., NewsNow).
- **_process_articles():** Parses raw HTML to extract title, link, time, and preliminary IOCs.
- **_extract_iocs(text):** Uses regex to find potential IOCs in text.
- **analyze_article(article):** Coordinates the analysis of IOCs found in an article.
- **monitor_and_analyze():** Main loop for continuous monitoring (primarily for standalone testing/CLI use).

3. ``web\_monitor.py`` (Web Application & Backend):

- FastAPI application setup.
- **monitor_thread():** Background task for continuous article fetching and broadcasting updates.
- WebSocket endpoint (/ws): Handles real-time communication with clients.
- HTTP endpoints:
 - /: Serves the main HTML page.
 - /recent_iocs: Provides recent IOCs from ThreatFox (demo/auxiliary feature).
 - /analyze: Accepts IOCs for manual analysis and returns HTML reports.
 - /analyze_article: Accepts article data for detailed analysis and returns HTML reports.
- **check_pcap_for_ioc(ioc):** Uses subprocess to run tshark for PCAP analysis.
- Manages recent_articles and active_connections (WebSockets).

4. ``rss\_monitor.py`` (RSS Feed Monitoring):

- Standalone script using feedparser to fetch and display new articles from an RSS feed (e.g., ThreatPost). Less integrated into the main web application flow but provides an alternative data source.

5. Templates & Static Files:

- templates/monitor.html: Main Jinja2 template for the web interface.
- static/: Directory for CSS, JavaScript, and images (if any).
- HTML/CSS for styling reports generated by ioc_analyzer.py.

6. Configuration & Cache:

- .env file: Stores API keys (e.g., AI_API).
- cache/mitre_attack_cache.json: Local cache of the MITRE ATT&CK enterprise data.

3.3. Technology Stack

- **Programming Language:** Python 3.8+
- **Web Framework:** FastAPI
- **Web Server:** Uvicorn
- **Templating Engine:** Jinja2
- **AI Integration:** Google Generative AI SDK (for Gemini API)
- **Threat Intelligence:** ThreatFox API (via requests)

- Justification:**

- ### 3.4. Data Flow Diagram



i Diagram Note

Zoom in to see the details.

4. IMPLEMENTATION STRATEGY

4.1. Development Methodology

An iterative and incremental development approach will be used, loosely following Agile principles.

1. **Core Module Development:** Focus on building and testing the core functionalities first (`ioc_analyzer.py`, `basic_cyber_monitor.py`).
2. **API Integration:** Thoroughly test integrations with Gemini AI and ThreatFox.
3. **Web Backend Development:** Develop FastAPI endpoints and WebSocket functionality.
4. **Web Frontend Development:** Create Jinja2 templates and basic frontend interactions.
5. **CLI Development:** Implement the command-line interface.
6. **Testing:** Unit tests for critical functions, integration tests for module interactions, and end-to-end testing for user workflows.
7. **Iteration:** Refine features and fix bugs based on testing and (simulated) user feedback.

Version control will be managed using Git, with regular commits and meaningful commit messages.

4.2. Modules and Phases

Phase 1: Core IOC Analysis Engine (Backend)

- **Module:** `ioc_analyzer.py`
 - Task: Setup Gemini AI and MITRE ATT&CK framework (caching).
 - Task: Implement `search_threatfox()` function.
 - Task: Implement `map_to_mitre()` function.
 - Task: Implement `analyze_with_gemini()` for IOC data.
 - Task: Implement `generate_report()` and `display_report()` (HTML output).
- **Module:** Basic CLI script for testing `ioc_analyzer.py`.
- **Testing:** Unit tests for each function in `ioc_analyzer.py`.

Phase 2: Data Collection & Initial Monitoring

- **Module:** `cyber_monitor.py`
 - Task: Implement `get_articles()` for NewsNow (or similar).
 - Task: Implement `_extract_iocs()` (regex-based initially).
 - Task: Integrate `cyber_monitor` with `ioc_analyzer` to analyze found IOCs.
- **Module:** `rss_monitor.py`
 - Task: Implement basic RSS feed fetching.
- **Testing:** Test data collection from sources, IOC extraction accuracy.

Phase 3: Web Application Backend & WebSocket

- **Module:** `web_monitor.py` (FastAPI)

- Task: Setup basic FastAPI app.
- Task: Implement `/analyze` endpoint for manual IOC submission, integrating with `ioc_analyzer`.
- Task: Implement `monitor_thread` for background article fetching using `CyberMonitor`.
- Task: Implement WebSocket (`/ws`) for broadcasting new articles.
- Task: Implement `/analyze_article` endpoint.
- Task: Implement `check_pcap_for_ioc()` and integrate into analysis endpoints.

- **Testing:** Test API endpoints with tools like Postman/curl. Test WebSocket communication.

Phase 4: Web Application Frontend

- **Module:** `templates/monitor.html` & static files
 - Task: Design and implement the main web page structure.
 - Task: Implement JavaScript for WebSocket message handling and dynamic UI updates.
 - Task: Implement forms for IOC submission and article analysis requests.
 - Task: Display analysis reports (HTML) returned from the backend.
- **Testing:** Browser-based testing of UI functionality and responsiveness.

Phase 5: Refinement, CLI, and Documentation

- **Module:** Enhanced CLI
 - Task: Develop a more user-friendly CLI based on `ioc_analyzer.py` and potentially `cyber_monitor.py`.
 - **Documentation:**
 - Task: Finalize `README.md` (or `README.rst`) with usage instructions.
 - Task: Add code comments and docstrings.
 - Task: Complete this Project Specification document.
 - **Testing:** End-to-end testing of all features (Web and CLI). User acceptance testing (simulated).
 - **Deployment:** Prepare `requirements.txt` and basic deployment notes.
-

5. PROJECT PLAN

5.1. Milestones and Deliverables

Table 3: Milestones and Deliverables

Milestone ID	Milestone Description	Estimated Completion Date	Deliverables
M1	Core IOC Analysis Engine Complete	Week 2	<code>ioc_analyzer.py</code> with ThreatFox, MITRE, Gemini integration; Basic CLI tester; Unit tests.
M2	Data Collection Modules Functional	Week 3	<code>cyber_monitor.py</code> and <code>rss_monitor.py</code> able to fetch and parse articles/feeds.
M3	Web Backend API and WebSocket Initial Setup	Week 5	FastAPI app with <code>/analyze</code> , <code>/analyze_article</code> endpoints, and <code>/ws</code> WebSocket working. PCAP check integrated.
M4	Web Frontend Basic UI and Interaction	Week 7	<code>monitor.html</code> displaying articles, allowing IOC/article analysis submission, and showing reports.
M5	Full Feature Integration and End-to-End Testing	Week 8	All components integrated; CLI tool refined; End-to-end test cases passed.
M6	Documentation, Final Review, and Project Submission Package	Week 9	Completed <code>project_spec.rst</code> , <code>README.rst</code> , code comments, test report, presentation slides (if applicable).

Note

The estimated completion dates are subject to change depending on how difficult the project is to implement.

5.2. Timeline

Detailed Weekly Plan (Illustrative):

- **Week 1-2 (M1):** Focus on `ioc_analyzer.py`. Setup, API integrations (Gemini, ThreatFox), MITRE caching. Basic CLI for testing.
- **Week 3 (M2):** Develop `cyber_monitor.py` and `rss_monitor.py`. Test article fetching and IOC extraction.
- **Week 4-5 (M3):** Develop FastAPI backend (`web_monitor.py`). Implement analysis endpoints, background monitoring thread, WebSocket, and PCAP integration.
- **Week 6-7 (M4):** Develop web frontend (`monitor.html`, JS). Implement UI for displaying data, submitting IOCs/articles, and showing reports.

- **Week 8 (M5):** Integrate all components. Conduct thorough end-to-end testing. Refine CLI.
- **Week 9 (M6):** Finalize all documentation, code comments. Prepare project for submission.

5.3. Resource Allocation

- **Primary Developer:** Daniel Vetrila
 - **Hardware:** Personal laptop/workstation meeting system requirements.
 - **Software:** Python, VS Code (or preferred IDE), Git, Web Browsers, tshark.
 - **Cloud Services/APIs:**
 - Google Gemini AI API (ensure API key is active and usage is within free tier limits if applicable, or budget is allocated).
 - ThreatFox API (public access).
 - **Time Commitment:** (10 hours per week).
-

6. METRICS FOR SUCCESS (FURPS+)

This section defines how the success and quality of the project will be evaluated, based on the FURPS+ model (Functionality, Usability, Reliability, Performance, Supportability + other relevant factors like Security).

6.1. Functionality

- **Metric:** All functional requirements (FR01-FR15) are implemented and pass corresponding test cases.
- **Target:** 100% of High priority FRs, 90% of Medium priority FRs.
- **Verification:** Test plan execution, demonstration of features.
- **Metric:** Accuracy of IOC extraction from articles (compared to manual analysis of a sample set).
- **Target:** >80% precision and recall for common IOC types (IPs, domains).
- **Verification:** Evaluation against a curated test set of articles.
- **Metric:** Correctness of MITRE ATT&CK mapping for a sample of known threat scenarios.
- **Target:** >85% correct mapping to relevant techniques.
- **Verification:** Expert review of mappings for test IOCs.

6.2. Usability

- **Metric:** Time taken for a new Cybersecurity Analyst user to understand the web interface and successfully analyze an IOC or article.
- **Target:** < 15 minutes for core tasks with minimal guidance.
- **Verification:** User testing with 2-3 representative users (can be peers).
- **Metric:** Clarity and actionability of the generated HTML reports.
- **Target:** Users rate reports as “Clear” or “Very Clear” and “Actionable” or “Very Actionable” on a 5-point Likert scale.
- **Verification:** Feedback survey from user testing.
- **Metric:** Ease of use of the CLI for basic IOC analysis.
- **Target:** Users can perform analysis with CLI using provided help/documentation within 5 minutes.
- **Verification:** User testing.

6.3. Reliability

- **Metric:** System uptime during a continuous 24-hour test run of the web application and monitoring components.
- **Target:** >99.5% uptime (allowing for minor, auto-recoverable glitches or external API downtimes).
- **Verification:** Monitored test run.
- **Metric:** Successful handling of external API errors (e.g., ThreatFox/Gemini unavailable).
- **Target:** System reports errors gracefully and continues operation for other functions, or retries appropriately. No crashes.

- **Verification:** Simulating API failures during testing.
- **Metric:** Data integrity of the MITRE ATT&CK cache.
- **Target:** Cache refresh mechanism works correctly; cached data matches source.
- **Verification:** Manual check post-refresh and during operations.

6.4. Performance

- **Metric:** Average response time for manual IOC analysis via web UI (from submission to report display).
- **Target:** < 30 seconds for 90% of requests (excluding unusually slow external API responses).
- **Verification:** Performance testing with a sample set of IOCs.
- **Metric:** Article processing time by CyberMonitor (from fetching to broadcasting via WebSocket).
- **Target:** Average < 10 seconds per new article batch.
- **Verification:** Logging and timing during test runs.
- **Metric:** Web page load time for the main monitoring dashboard.
- **Target:** < 3 seconds on a typical broadband connection.
- **Verification:** Browser developer tools.
- **Metric:** CPU and memory usage under normal load (e.g., 10 active WebSocket clients, monitoring running).
- **Target:** Within acceptable limits of the target deployment environment (e.g., <50% average CPU, <1GB RAM for the application).
- **Verification:** System monitoring tools.

6.5. Supportability (Maintainability)

- **Metric:** Code complexity (e.g., McCabe cyclomatic complexity for key functions).
- **Target:** Average complexity score < 10 for critical functions.
- **Verification:** Static analysis tools (if available) or manual review.
- **Metric:** Percentage of code covered by unit tests.
- **Target:** >60% for core logic in `ioc_analyzer.py`.
- **Verification:** Code coverage reports (e.g., `coverage.py`).
- **Metric:** Quality and completeness of code comments and documentation (this document, README).
- **Target:** All major functions and classes have docstrings; complex logic is commented. README is comprehensive.
- **Verification:** Manual review.
- **Metric:** Ease of setting up the development environment and running the project.
- **Target:** New developer can set up and run the project within 30 minutes using `requirements.txt` and README.
- **Verification:** Test by a peer.

6.6. Security (Added Metric)

- **Metric:** Secure handling of API keys.
 - **Target:** API keys are not hardcoded in source files and are loaded from environment variables.
 - **Verification:** Code review.
 - **Metric:** Basic protection against common web vulnerabilities (e.g., XSS in report display if user-controlled IOCs were mishandled, though less likely with current design).
 - **Target:** Inputs are appropriately handled/sanitized if they were to be directly rendered without templating engine's auto-escaping. FastAPI's features are utilized.
 - **Verification:** Code review focusing on input handling and output encoding (Jinja2 auto-escapes by default).
-

7. CONCLUSION

The AI-Powered Cyber Incident Monitoring Tool project aims to deliver a valuable asset for cybersecurity professionals by automating and enhancing the process of threat detection and analysis. By successfully meeting the objectives and requirements outlined in this document, the project will provide a robust, user-friendly, and insightful platform. The defined implementation strategy, project plan, and metrics for success will guide the development and ensure a high-quality outcome.